

Guide d'installation et de lancement de l'application SGRH Intelligent

MANSOUR Aymane FOUKOU Issa BADIA Salahedine

Année universitaire 2025–2026

Table des matières

1	Introduction	3
2	Prérequis logiciels	4
2.1	Logiciels à installer	4
2.2	Vérification de l'installation	4
2.3	Configuration de la variable PATH	5
2.3.1	Sous Windows	5
2.3.2	Sous macOS / Linux	5
3	Récupération du projet	6
3.1	Téléchargement du code source	6
3.1.1	Depuis une archive ZIP	6
3.1.2	Depuis Git (si disponible)	6
3.2	Structure du projet	6
4	Configuration de la base de données	8
4.1	Démarrage de MySQL	8
4.1.1	Sous Windows	8
4.1.2	Sous macOS	8
4.1.3	Sous Linux	8
4.2	Création de la base de données	8
4.3	Configuration du mot de passe MySQL	9
5	Lancement des services	10
5.1	Étape 1 — Microservice IA (port 8000)	10
5.2	Étape 2 — Backend Spring Boot (port 8080)	11
5.3	Étape 3 — Frontend React (port 3000)	11
5.4	Résumé de l'ordre de lancement	12
6	Interface de connexion (Login)	13
6.1	Accès à la page de connexion	13
6.2	Comptes de connexion disponibles	13
6.3	Rôles et accès	14
6.4	Procédure de connexion	14
6.5	Connexion rapide via la page autologin	15
7	Lancement sur différentes machines	16
7.1	Cas 1 : Tout sur la même machine (développement)	16
7.2	Cas 2 : Backend et Frontend sur des machines différentes	16

7.3	Cas 3 : Accès depuis un autre poste sur le réseau local	17
7.4	Cas 4 : Déploiement en production	17
8	Résolution des problèmes courants	19
9	Conclusion	20

Chapitre 1

Introduction

Ce guide explique comment installer, configurer et lancer l'application **SGRH Intelligent** sur n'importe quelle machine (Windows, macOS ou Linux). Il s'adresse à toute personne souhaitant déployer le projet, que ce soit un développeur, un membre du jury de soutenance ou un utilisateur final. L'objectif est de fournir une procédure claire, progressive et reproductible, permettant de passer d'une machine vierge à une application fonctionnelle en quelques étapes.

Le projet SGRH Intelligent repose sur une architecture distribuée composée de plusieurs services indépendants qui communiquent entre eux via des protocoles réseau standards (HTTP/REST). Chaque service est responsable d'une partie précise du système : la base de données conserve les informations, le backend expose la logique métier, le microservice IA effectue les analyses intelligentes et le frontend affiche les interfaces utilisateur dans le navigateur.

Cette architecture implique que plusieurs programmes doivent fonctionner simultanément sur la machine pour que l'application soit opérationnelle. Si l'un des services est arrêté, certaines fonctionnalités peuvent devenir indisponibles. Par exemple, si MySQL n'est pas démarré, le backend ne pourra pas lire ni écrire les données. Si le microservice IA est éteint, les fonctions d'analyse et de détection d'anomalies seront inaccessibles, mais le reste de l'application continuera de fonctionner normalement.

L'application est composée de quatre services qui doivent fonctionner simultanément :

1. **MySQL** — base de données (port 3306)
2. **Microservice IA** — intelligence artificielle (port 8000)
3. **Backend Spring Boot** — logique métier et API REST (port 8080)
4. **Frontend React** — interface utilisateur (port 3000)

Chapitre 2

Prérequis logiciels

2.1 Logiciels à installer

L'application utilise plusieurs technologies complémentaires, chacune nécessitant un environnement d'exécution spécifique. Le frontend est développé en React avec JavaScript, ce qui nécessite Node.js pour gérer les dépendances et compiler le code. Le backend est développé en Java avec le framework Spring Boot, ce qui nécessite un JDK (Java Development Kit) version 21 et l'outil de construction Maven. Le microservice IA est écrit en Python avec le framework FastAPI, ce qui nécessite un interpréteur Python version 3.11 ou supérieure. Enfin, les données sont stockées dans une base MySQL, qui nécessite l'installation du serveur de base de données.

Chaque logiciel joue un rôle précis dans le fonctionnement global de l'application. Il est important de respecter les versions minimales indiquées, car certaines fonctionnalités du code dépendent de caractéristiques introduites dans ces versions spécifiques. Par exemple, le backend utilise des fonctionnalités de Java 21 qui ne sont pas disponibles dans les versions antérieures.

Avant de lancer l'application, les logiciels suivants doivent être installés sur la machine :

Logiciel	Version minimale	Téléchargement
Node.js	18.x	https://nodejs.org/
Java JDK	21	https://adoptium.net/
Apache Maven	3.9.x	https://maven.apache.org/download.cgi
Python	3.11	https://www.python.org/downloads/
MySQL Server	8.x	https://dev.mysql.com/downloads/mysql/
Tesseract OCR	5.x (optionnel)	https://github.com/tesseract-ocr/tesseract

2.2 Vérification de l'installation

Après l'installation de chaque logiciel, il est essentiel de vérifier que la commande correspondante est accessible depuis le terminal. Cette vérification permet de s'assurer que le programme a été correctement installé et que le système d'exploitation sait où

le trouver. En effet, certains installateurs n'ajoutent pas automatiquement le chemin de l'exécutable dans la variable d'environnement `PATH`, ce qui peut empêcher son utilisation depuis un terminal.

Pour vérifier que chaque outil est bien installé, ouvrir un terminal (PowerShell sous Windows, Terminal sous macOS, ou un émulateur de terminal sous Linux) et exécuter les commandes suivantes :

```
node --version      (doit afficher v18.x ou supérieur)
java --version      (doit afficher 21.x)
mvn --version       (doit afficher 3.9.x)
python --version    (doit afficher 3.11.x ou supérieur)
mysql --version     (doit afficher 8.x)
```

Si une commande n'est pas reconnue (message du type « n'est pas reconnu comme une commande interne » sous Windows ou « command not found » sous Linux/macOS), cela signifie que le logiciel n'est pas installé ou que son chemin n'est pas dans la variable d'environnement `PATH`. La section suivante explique comment configurer cette variable.

2.3 Configuration de la variable `PATH`

La variable d'environnement `PATH` est une liste de dossiers dans lesquels le système d'exploitation recherche les programmes exécutables lorsqu'une commande est tapée dans le terminal. Si le dossier contenant un programme n'est pas dans cette liste, le système ne pourra pas le trouver et affichera une erreur. La configuration du `PATH` est une étape indispensable pour pouvoir utiliser Java, Maven, Python et MySQL depuis le terminal.

2.3.1 Sous Windows

1. Ouvrir **Paramètres** → **Système** → **Informations système** → **Paramètres système avancés**
2. Cliquer sur **Variables d'environnement**
3. Dans **Variables système**, sélectionner `Path` et cliquer sur **Modifier**
4. Ajouter les chemins vers les dossiers `bin` de Java, Maven, Python et MySQL

Exemple de chemins sous Windows :

```
C:\Program Files\Eclipse Adoptium\jdk-21\bin
C:\Users\<utilisateur>\maven\bin
C:\Program Files\MySQL\MySQL Server 8.4\bin
C:\Users\<utilisateur>\AppData\Local\Programs\Python\Python311
```

2.3.2 Sous macOS / Linux

Ajouter dans le fichier `~/.bashrc` ou `~/.zshrc` :

```
export JAVA_HOME=/usr/lib/jvm/java-21
export PATH=$JAVA_HOME/bin:$PATH
export PATH=$HOME/apache-maven-3.9.6/bin:$PATH
```

Puis exécuter `source ~/.bashrc` pour appliquer les modifications.

Chapitre 3

Récupération du projet

3.1 Téléchargement du code source

Le code source du projet contient l'ensemble des fichiers nécessaires au fonctionnement de l'application : le code Java du backend, le code JavaScript/React du frontend, le code Python du microservice IA, les scripts SQL pour la base de données et les fichiers de documentation. L'ensemble de ces fichiers doit être présent sur la machine avant de pouvoir lancer les services.

Le projet peut être récupéré de deux manières :

3.1.1 Depuis une archive ZIP

Si le projet est fourni sous forme d'archive :

1. Extraire l'archive `PFE_Slah.zip` dans un dossier de votre choix
2. Le dossier extrait contient les sous-dossiers : `frontend`, `backend`, `microservice-ia`, `database`, `documentation`

3.1.2 Depuis Git (si disponible)

```
git clone <url_du_depot> PFE_Slah
cd PFE_Slah
```

3.2 Structure du projet

Une fois le projet récupéré, le dossier principal contient plusieurs sous-dossiers, chacun correspondant à un composant de l'application. Cette organisation facilite la navigation dans le code et permet de travailler sur un composant sans affecter les autres. Chaque sous-dossier est autonome et possède ses propres fichiers de configuration et de dépendances.

```
PFE_Slah/
  backend/          --> Application Spring Boot (Java)
  frontend/         --> Application React (JavaScript)
  microservice-ia/  --> Microservice FastAPI (Python)
  database/         --> Scripts SQL (schema + seed)
  documentation/    --> Documents LaTeX
```

Le dossier `backend` contient le code source Java, le fichier de configuration `application.yml` et le fichier de dépendances Maven `pom.xml`. Le dossier `frontend` contient les composants React, les fichiers de style CSS et le fichier `package.json` qui liste les dépendances JavaScript. Le dossier `microservice-ia` contient le fichier principal `main.py` et le fichier `requirements.txt` listant les bibliothèques Python. Le dossier `database` contient les scripts SQL de création de la base et d'insertion des données de démonstration.

Chapitre 4

Configuration de la base de données

4.1 Démarrage de MySQL

MySQL est le système de gestion de base de données relationnelle utilisé par l'application. Il stocke toutes les informations : les comptes utilisateurs, les données des employés, les demandes de congés, les présences, les fiches de paie, les documents et les anomalies détectées par l'IA. Sans MySQL, le backend ne peut ni lire ni écrire de données, ce qui rend l'application inutilisable.

Le serveur MySQL doit être démarré avant le backend. La méthode de démarrage dépend du système d'exploitation et de la manière dont MySQL a été installé (service système, installation manuelle, etc.).

4.1.1 Sous Windows

Si MySQL est installé en tant que service Windows (cas le plus courant avec l'installateur officiel), il peut être démarré avec la commande suivante dans un terminal administrateur :

```
net start mysql84
```

Sinon, lancer le serveur manuellement :

```
mysqld --console
```

4.1.2 Sous macOS

```
brew services start mysql
```

4.1.3 Sous Linux

```
sudo systemctl start mysql
```

4.2 Création de la base de données

Une fois MySQL démarré, il faut créer la base de données et ses tables. Le projet fournit deux scripts SQL prêts à l'emploi. Le premier script, `schema.sql`, contient les instructions `CREATE DATABASE` et `CREATE TABLE` qui définissent la structure complète de la base : les

tables, les colonnes, les types de données, les clés primaires, les clés étrangères et les contraintes. Le second script, `seed.sql`, contient des instructions `INSERT` qui ajoutent des données de démonstration dans les tables.

Ces données de démonstration sont essentielles pour pouvoir tester l'application immédiatement après l'installation. Elles incluent notamment les comptes utilisateurs avec des mots de passe hashés en BCrypt, des employés fictifs, des demandes de congés, des enregistrements de présence et des fiches de paie.

Ouvrir un terminal et exécuter les scripts fournis :

```
mysql -u root -p < database/schema.sql
mysql -u root -p sgrh_intelligent < database/seed.sql
```

Remarque importante : si la base `sgrh_intelligent` existe déjà (par exemple après une installation précédente), le script `schema.sql` la supprime et la recrée. Toutes les données existantes seront donc perdues. Si vous souhaitez conserver vos données, ne réexécutez pas ce script.

4.3 Configuration du mot de passe MySQL

Le backend Spring Boot doit connaître les identifiants de connexion à MySQL pour pouvoir établir la connexion au démarrage. Ces identifiants sont définis dans le fichier de configuration `application.yml`. Si le mot de passe MySQL de votre machine est différent de celui configuré dans ce fichier, le backend affichera une erreur de type `Access denied` au démarrage et ne pourra pas fonctionner.

Le backend attend la configuration suivante (fichier `backend/src/main/resources/application.yml`) :

```
spring:
  datasource:
    url: jdbc:mysql://localhost:3306/sgrh_intelligent
    username: root
    password: root
```

Si votre mot de passe MySQL est différent de `root`, modifiez la valeur du champ `password` dans ce fichier avant de lancer le backend. Par exemple, si votre mot de passe MySQL est vide, remplacez `root` par une chaîne vide. Si vous utilisez un autre utilisateur que `root`, modifiez également le champ `username`.

Chapitre 5

Lancement des services

Important : les services doivent être lancés dans un ordre précis, car chaque service dépend du précédent. Le backend a besoin de MySQL pour accéder aux données, et il a besoin du microservice IA pour les fonctionnalités d'analyse. Le frontend a besoin du backend pour afficher les informations. Si un service est lancé avant ses dépendances, il affichera des erreurs de connexion.

Chaque service doit fonctionner dans un terminal séparé. Cela signifie que vous devez ouvrir au minimum trois fenêtres de terminal simultanément (une pour le microservice IA, une pour le backend et une pour le frontend). Chaque terminal restera actif tant que le service correspondant fonctionne. Fermer un terminal arrête le service qu'il contient.

5.1 Étape 1 — Microservice IA (port 8000)

Le microservice IA est un serveur Python basé sur FastAPI. Il fournit des endpoints d'analyse de données : détection d'anomalies comportementales à l'aide de l'algorithme Isolation Forest, et segmentation des profils d'employés à l'aide de l'algorithme K-Means. Le backend Spring Boot communique avec ce service via des requêtes HTTP lorsque l'utilisateur demande une analyse.

La commande `pip install -r requirements.txt` télécharge et installe les bibliothèques Python nécessaires (FastAPI, Uvicorn, Scikit-learn, Pandas, NumPy). Cette opération n'est nécessaire qu'une seule fois, sauf si les dépendances changent. La commande `uvicorn` démarre le serveur web qui écoute sur le port 8000.

Ouvrir un terminal dans le dossier `microservice-ia` :

```
cd microservice-ia
pip install -r requirements.txt
python -m uvicorn main:app --reload --host 0.0.0.0 --port 8000
```

Sous Windows avec Python 3.11 installé à côté d'autres versions :

```
cd microservice-ia
py -3.11 -m pip install -r requirements.txt
py -3.11 -m uvicorn main:app --reload --host 0.0.0.0 --port 8000
```

Vérification : Ouvrir <http://localhost:8000/docs> dans le navigateur. La documentation Swagger doit s'afficher.

5.2 Étape 2 — Backend Spring Boot (port 8080)

Le backend Spring Boot est le composant central de l'application. Il reçoit les requêtes HTTP envoyées par le frontend, vérifie l'authentification via les tokens JWT, applique les règles de gestion métier (par exemple, décrémenter le solde de congé lors d'une validation) et communique avec la base de données MySQL pour lire ou écrire les informations.

La commande `mvn spring-boot:run` compile le code Java, résout les dépendances définies dans le fichier `pom.xml`, puis démarre le serveur Tomcat intégré sur le port 8080. La première exécution peut prendre plusieurs minutes car Maven télécharge toutes les bibliothèques nécessaires depuis les dépôts en ligne. Les exécutions suivantes seront plus rapides car les dépendances sont mises en cache localement.

Ouvrir un second terminal dans le dossier `backend` :

```
cd backend
mvn spring-boot:run
```

Le démarrage prend environ 10 à 15 secondes. Le message suivant doit apparaître :

```
Started SgrhApplication in X seconds
Tomcat started on port 8080
```

Vérification : Ouvrir <http://localhost:8080/api/dashboard/stats> dans le navigateur. Une réponse JSON doit s'afficher.

5.3 Étape 3 — Frontend React (port 3000)

Le frontend React est l'interface graphique de l'application. Il s'exécute dans le navigateur web de l'utilisateur et communique avec le backend via des requêtes HTTP (API REST). Le serveur de développement Vite compile le code React en temps réel et fournit un rechargement automatique : chaque modification du code source est immédiatement visible dans le navigateur sans avoir à recharger manuellement la page.

La commande `npm install` télécharge toutes les bibliothèques JavaScript définies dans le fichier `package.json` (React, React Router, Axios, Recharts, TailwindCSS, etc.) et les place dans le dossier `node_modules`. Cette opération n'est nécessaire qu'une seule fois, ou lorsque les dépendances sont mises à jour. La commande `npm run dev` lance le serveur Vite sur le port 3000.

Ouvrir un troisième terminal dans le dossier `frontend` :

```
cd frontend
npm install
npm run dev
```

La première fois, `npm install` télécharge les dépendances JavaScript (peut prendre quelques minutes selon la connexion internet). Le message suivant doit apparaître :

```
VITE ready in XXX ms
Local: http://localhost:3000/
```

Vérification : Ouvrir <http://localhost:3000> dans le navigateur. La page de connexion doit s'afficher.

5.4 Résumé de l'ordre de lancement

Ordre	Service	Port	Commande
1	MySQL	3306	Service système
2	Microservice IA	8000	<code>uvicorn main:app</code>
3	Backend Spring Boot	8080	<code>mvn spring-boot:run</code>
4	Frontend React	3000	<code>npm run dev</code>

Chapitre 6

Interface de connexion (Login)

6.1 Accès à la page de connexion

L'interface de connexion est la première page affichée lorsqu'un utilisateur accède à l'application. Elle joue un rôle de sécurité fondamental, car elle empêche tout accès non autorisé aux données de l'application. Seuls les utilisateurs disposant d'un compte valide peuvent se connecter et accéder aux fonctionnalités internes.

La page de connexion est composée de deux parties visuelles. La partie gauche présente le nom de l'application, une brève description et les principales fonctionnalités (IA, OCR, RH). La partie droite contient le formulaire de connexion proprement dit. Cette disposition en deux colonnes permet d'informer le visiteur sur la nature de l'application tout en lui offrant un accès direct au formulaire.

Une fois tous les services lancés, ouvrir un navigateur web (Chrome, Edge, Firefox) et accéder à l'adresse suivante :

<http://localhost:3000/login>

La page de connexion s'affiche avec :

- Un champ **Email** pour saisir l'adresse de l'utilisateur
- Un champ **Mot de passe** avec possibilité d'afficher ou masquer le texte
- Un bouton **Se connecter**
- Un mode **Démonstration** avec trois boutons (Admin, RH, Employé)

6.2 Comptes de connexion disponibles

Lors de l'exécution du script `seed.sql`, plusieurs comptes utilisateurs sont créés dans la base de données. Chaque compte est associé à un rôle qui détermine les pages et les fonctionnalités accessibles après connexion. Les mots de passe sont stockés sous forme hashée (BCrypt) dans la base de données, ce qui signifie qu'ils ne sont pas lisibles en clair, même par un administrateur de la base.

Les comptes de démonstration partagent tous le même mot de passe (`password123`) pour simplifier les tests. En environnement de production, chaque utilisateur devrait disposer d'un mot de passe unique et sécurisé.

Les comptes suivants sont créés automatiquement par le script `seed.sql` :

Email	Mot de passe	Rôle
admin@sgrh.ma	password123	ADMIN
rh@sgrh.ma	password123	RH
ahmed.benali@sgrh.ma	password123	EMPLOYE

6.3 Rôles et accès

Le système de rôles est un mécanisme de contrôle d'accès qui permet de restreindre les fonctionnalités selon le profil de l'utilisateur. Ce mécanisme est appliqué à deux niveaux : côté frontend (le menu latéral n'affiche que les pages autorisées) et côté backend (les endpoints API vérifient le rôle avant d'exécuter une opération). Cette double vérification garantit qu'un utilisateur ne peut pas contourner les restrictions en manipulant directement les requêtes HTTP.

Le rôle **ADMIN** dispose de tous les droits et peut notamment gérer les comptes utilisateurs. Le rôle **RH** est destiné aux responsables des ressources humaines qui gèrent le quotidien opérationnel. Le rôle **EMPLOYE** offre un accès limité aux informations personnelles.

Rôle	Modules accessibles
ADMIN	Tableau de bord, Employés, Congés, Présences, Paie, Documents, Analyse IA, Utilisateurs
RH	Tableau de bord, Employés, Congés, Présences, Paie, Documents, Analyse IA
EMPLOYE	Tableau de bord, Congés, Présences, Documents

6.4 Procédure de connexion

La procédure de connexion suit un processus technique précis en arrière-plan. Lorsque l'utilisateur clique sur « Se connecter », le frontend envoie une requête POST à l'endpoint `/api/auth/login` du backend avec l'email et le mot de passe. Le backend vérifie les identifiants en comparant le mot de passe saisi avec le hash BCrypt stocké en base de données. Si la correspondance est confirmée, le backend génère un token JWT (JSON Web Token) signé contenant l'identité et le rôle de l'utilisateur, puis le renvoie au frontend.

Le frontend stocke ce token dans le `localStorage` du navigateur. À chaque requête ultérieure vers l'API, ce token est automatiquement ajouté dans l'en-tête HTTP **Authorization**. Le backend vérifie la validité du token à chaque requête, ce qui permet de maintenir la session sans stocker d'état côté serveur.

1. Ouvrir <http://localhost:3000> dans le navigateur
2. Saisir l'email : `admin@sgrh.ma`
3. Saisir le mot de passe : `password123`
4. Cliquer sur **Se connecter**
5. L'application redirige vers le **Tableau de bord**

En cas d'erreur « Identifiants incorrects », vérifier que :

- Le backend est bien lancé et accessible sur le port 8080

- La base de données MySQL est démarrée
- Le script `seed.sql` a été exécuté correctement
- L'email et le mot de passe sont correctement saisis (attention aux espaces)

6.5 Connexion rapide via la page autologin

Une page de connexion automatique est disponible à l'adresse :

<http://localhost:3000/autologin.html>

Cette page permet de se connecter directement en tant qu'administrateur en un seul clic, sans avoir à saisir manuellement les identifiants. Elle fonctionne en appelant directement l'API de login via JavaScript, en récupérant le token JWT retourné, en le stockant dans le `localStorage`, puis en redirigeant automatiquement vers le tableau de bord. Cette page est particulièrement utile lors des démonstrations ou des soutenances, car elle permet de montrer rapidement l'application sans perdre de temps sur la saisie des identifiants.

Chapitre 7

Lancement sur différentes machines

7.1 Cas 1 : Tout sur la même machine (développement)

C'est le cas le plus simple et le plus courant pendant la phase de développement ou de présentation. Tous les services (MySQL, microservice IA, backend et frontend) tournent sur la même machine physique et communiquent via l'adresse `localhost` (qui correspond à l'adresse IP `127.0.0.1`). Il suffit de suivre les étapes du chapitre précédent. L'application est accessible à <http://localhost:3000>.

Cette configuration ne nécessite aucune modification des fichiers de configuration. Les adresses par défaut (`localhost:8080` pour le backend, `localhost:8000` pour le microservice IA) sont déjà correctement définies dans le code source.

7.2 Cas 2 : Backend et Frontend sur des machines différentes

Dans certaines situations, il peut être utile de séparer le backend et le frontend sur deux machines distinctes. Par exemple, le backend peut être hébergé sur un serveur plus puissant disposant de plus de mémoire et d'un accès rapide à la base de données, tandis que le frontend peut être lancé sur la machine de l'utilisateur. Cette configuration est courante dans les environnements de test ou de pré-production.

Dans ce cas, le frontend doit savoir où se trouve le backend. Par défaut, il envoie ses requêtes à `localhost:8080`, ce qui ne fonctionne que si les deux services sont sur la même machine. Il faut donc modifier l'URL de base de l'API dans le fichier de configuration du frontend.

Si le backend est hébergé sur une machine serveur (par exemple `192.168.1.100`) :

1. Sur la machine serveur : lancer MySQL, le microservice IA et le backend
2. Modifier le fichier `frontend/src/services/api.js` pour pointer vers l'adresse du serveur :

```
const API = axios.create({
  baseURL: 'http://192.168.1.100:8080/api'
})
```

3. Sur la machine client : lancer le frontend avec `npm run dev`
4. Accéder à <http://localhost:3000> depuis le client

7.3 Cas 3 : Accès depuis un autre poste sur le réseau local

Il est possible d'accéder à l'application depuis un autre ordinateur connecté au même réseau local (Wi-Fi ou câble Ethernet). Cela peut être utile, par exemple, pour faire une démonstration sur un écran de projection connecté à un autre poste, ou pour permettre à plusieurs personnes de tester l'application simultanément.

Par défaut, le serveur Vite n'écoute que sur `localhost`, ce qui signifie qu'il refuse les connexions venant d'autres machines. Pour autoriser les connexions externes, il faut lancer le frontend avec l'option `-host 0.0.0.0`, qui indique au serveur d'écouter sur toutes les interfaces réseau de la machine.

Si le frontend tourne sur une machine et qu'on souhaite y accéder depuis un autre poste du même réseau :

1. Lancer le frontend avec l'option `host` :

```
npm run dev -- --host 0.0.0.0
```

2. Depuis l'autre poste, ouvrir le navigateur et accéder à :

```
http://<adresse_ip_de_la_machine>:3000
```

Pour connaître l'adresse IP de la machine sous Windows :

```
ipconfig
```

Sous macOS / Linux :

```
ifconfig    ou    ip addr
```

7.4 Cas 4 : Déploiement en production

Le déploiement en production désigne la mise en ligne de l'application sur un serveur accessible de manière permanente, par exemple sur un serveur cloud ou un serveur d'entreprise. Dans ce contexte, les serveurs de développement (Vite pour le frontend, `mvn spring-boot:run` pour le backend) ne sont plus adaptés, car ils sont optimisés pour le développement et non pour la performance ou la sécurité en production.

Pour un déploiement en production, il est recommandé de :

- Construire le frontend avec `npm run build` : cette commande compile le code React en fichiers HTML, CSS et JavaScript optimisés, qui peuvent ensuite être servis par un serveur web comme Nginx ou Apache.
- Packager le backend en JAR avec `mvn package` : cette commande produit un fichier `.jar` autonome contenant l'application et toutes ses dépendances. Il peut être exécuté avec `java -jar sgrh-0.0.1-SNAPSHOT.jar`.
- Utiliser un serveur MySQL dédié avec un mot de passe sécurisé : en production, le mot de passe `root` ne doit jamais être utilisé. Il faut créer un utilisateur MySQL spécifique avec des droits limités.

- Configurer un reverse proxy (Nginx) pour rediriger les requêtes : le reverse proxy permet de servir le frontend et le backend depuis un même domaine, en redirigeant les requêtes `/api/` vers le port 8080 du backend.
- Activer HTTPS avec un certificat SSL pour sécuriser les communications entre le navigateur et le serveur.

Chapitre 8

Résolution des problèmes courants

Lors de l'installation ou du lancement de l'application, certains problèmes peuvent survenir. Le tableau ci-dessous présente les erreurs les plus fréquemment rencontrées ainsi que les solutions à appliquer. Dans la majorité des cas, ces problèmes sont liés à un logiciel manquant, un service non démarré ou une mauvaise configuration.

En cas de doute, il est recommandé de vérifier dans l'ordre : (1) que MySQL est démarré, (2) que le backend affiche le message « Started SgrhApplication » sans erreur, (3) que le frontend affiche « VITE ready » dans son terminal, et (4) que le navigateur accède bien à `http://localhost:3000`.

Problème	Solution
<code>mvn</code> non reconnu	Installer Maven et ajouter son dossier <code>bin</code> au PATH
<code>node</code> non reconnu	Installer Node.js depuis https://nodejs.org/
Port 3306 injoignable	Vérifier que MySQL est démarré
Port 8080 injoignable	Vérifier que le backend Spring Boot est lancé sans erreur
Identifiants incorrects	Vérifier que <code>seed.sql</code> a été exécuté et que le mot de passe MySQL dans <code>application.yml</code> est correct
Page blanche sur le frontend	Vérifier la console du navigateur (F12) pour identifier l'erreur JavaScript
CORS error	Vérifier que le backend autorise les requêtes depuis le port 3000 (configuré dans <code>SecurityConfig.java</code>)
Microservice IA non disponible	Vérifier que Python et les dépendances sont installés, et que le service tourne sur le port 8000

Chapitre 9

Conclusion

Ce guide a présenté de manière détaillée l'ensemble des étapes nécessaires pour installer, configurer et lancer l'application SGRH Intelligent sur une machine. Depuis l'installation des prérequis logiciels jusqu'à la connexion à l'interface, chaque étape a été expliquée avec suffisamment de détail pour être reproductible par un utilisateur n'ayant pas participé au développement du projet.

L'architecture modulaire du projet (frontend React, backend Spring Boot, microservice IA FastAPI, base de données MySQL) offre une grande flexibilité de déploiement. Les quatre services peuvent fonctionner sur une seule machine pour le développement et les démonstrations, ou être répartis sur plusieurs serveurs dans un environnement de production. Cette séparation des responsabilités facilite également la maintenance : chaque composant peut être mis à jour, redémarré ou remplacé indépendamment des autres.

Les identifiants de démonstration fournis permettent de tester immédiatement l'application avec les trois rôles disponibles (Administrateur, Responsable RH, Employé) et d'explorer l'ensemble des fonctionnalités proposées. Ce guide peut être utilisé comme annexe du rapport de Projet de Fin d'Études ou comme documentation autonome à remettre aux membres du jury ou aux futurs utilisateurs de l'application.